Unidad 4: Comunicación en Sistemas Distribuidos

1. Introducción

La diferencia más importante entre un **sistema distribuido** y un sistema de único procesador es la **comunicación entre procesos**:

- En un sistema de único procesador se asume el uso de **memoria compartida**.
- En un sistema distribuido **no existe memoria compartida**; toda la comunicación se basa en **pasaje de mensajes** y debe apegarse a protocolos de comunicación.

Los diferentes mecanismos de comunicación se caracterizan por los siguientes factores:

- **Rendimiento**: latencia, ancho de banda, etc.
- **Escalabilidad**: número de elementos activos en el sistema.
- **Fiabilidad**: pérdida de mensajes y tolerancia a fallos.
- **Seguridad**: cifrado, certificación de identidad.
- **Movilidad**: soporte para equipos móviles.
- **Calidad de Servicio (QoS)**: reserva y garantía de anchos de banda.
- **Comunicación en grupo**: soporte para multicast.

2. Características Deseables de la Comunicación

Un buen mecanismo de comunicación para sistemas distribuidos debería cumplir con las siguientes propiedades:

- **Simplicidad**: el programador no debe preocuparse por aspectos de red o de sistema subyacentes.
- **Semántica uniforme**: las comunicaciones locales y remotas deberían comportarse de la misma manera desde la perspectiva del programador.
- **Eficiencia**: reducción del número de mensajes intercambiados.
- **Confiabilidad**: uso de mecanismos como timeouts para detectar fallos.
- **Flexibilidad**: control de flujo entre procesos cooperativos, soportando tanto mensajes sincrónicos como asincrónicos.
- **Seguridad**: encriptación del mensaje y autenticación de emisor y receptor.
- **Portabilidad**: capacidad de trabajar sobre plataformas heterogéneas, compatibilizando la representación de los datos.

3. Niveles de Abstracción en la Comunicación

Nivel 1 — Paso de Mensajes Puro

Es el paradigma más sencillo de comunicación a nivel de procesos. Dos procesos se comunican intercambiando mensajes básicos entre ellos: uno envía una solicitud y el otro puede generar una respuesta.

Requiere cuatro operaciones fundamentales:

- **Conexión** y **desconexión** (gestión de la comunicación).
- **Enviar** y **recibir** (transferencia de la información).

La programación basada en **sockets** está basada en este paradigma. Es también la base del modelo cliente/servidor.

Nivel 2 — Cliente / Servidor

Se basa en asignar dos funcionalidades distintas a cada participante:

- **Servidor**: proporciona servicios y espera pasivamente la llegada de peticiones.
- **Cliente**: realiza solicitudes activamente.

La mayor parte de los protocolos de Internet siguen este paradigma. Sus formas de implementación más comunes son: RPC, RMI y Web Services.

4. Protocolo Solicitud – Respuesta

Para evitar el gasto excesivo que implican los protocolos orientados a conexión, las comunicaciones se basan en un protocolo **solicitud/respuesta sin conexión**:

- El **Emisor** envía un mensaje de solicitud para pedir cierto servicio.
- El **Receptor** realiza el trabajo y devuelve los datos o un código de error.

Solo se requieren dos primitivas:

- `send(dest, &mptr)`: envía el mensaje y bloquea al proceso emisor.
- `receive(addr, &mptr)`: recibe el mensaje.

Problemas potenciales y sus soluciones:

Problema	Solución
Los mensajes se pueden perder en la red	Temporizadores (timeouts), ACKs y retransmisiones
El mensaje llegó bien pero el ACK se pierde, generando duplicados	Números de secuencia para identificar mensajes ya procesados

5. Consideraciones del IPC

El diseño de la comunicación entre procesos (IPC) plantea una serie de preguntas que deben responderse:

- ¿Quién envía? ¿Quién recibe?
- ¿Hay uno o varios receptores?
- ¿Necesita el send esperar una respuesta?
- ¿Está garantizada la entrega del mensaje al receptor?
- ¿Qué se debe hacer cuando falla el sitio o el enlace?
- ¿Qué sucede si el receptor no está listo para recibir el mensaje?
- Si hay varios mensajes esperando en el receptor, ¿puede éste cambiar el orden de procesamiento?

6. Aspectos Importantes: Sincronización y Representación

Sincronización

Como en un sistema distribuido **no existe un reloj global**, es necesario asegurar la ocurrencia causal de eventos. Esto significa que el sistema debe poder determinar qué evento ocurrió antes que otro sin depender de un reloj compartido y centralizado.

Representación de Datos y Marshalling

Tanto el emisor como el receptor deben coincidir en la interpretación de los bytes transmitidos. Este problema es especialmente relevante en sistemas heterogéneos donde los nodos pueden tener arquitecturas distintas (big-endian vs. little-endian, distintos tamaños de tipos de datos, distintas representaciones de punto flotante).

Marshaling (o serialización): proceso de ensamblado de datos de forma que puedan ser convenientemente transmitidos en un mensaje. Implica la traducción de los valores locales a una representación externa estandarizada.

Representación Externa de Datos (Coulouris, Cap. 4.3)

El problema de la representación externa surge porque diferentes arquitecturas de hardware pueden representar los mismos datos de formas incompatibles. La solución es acordar un **formato externo común** (formato de red) al que todos los nodos convierten sus datos antes de transmitir, y desde el cual convierten al recibirlos.

Enfoques principales

XDR (eXternal Data Representation — Sun Microsystems): Uno de los formatos más tempranos. Define representaciones para tipos primitivos (enteros de 4 bytes big-endian, punto flotante IEEE 754, strings con longitud prefijada, etc.). Fue la base de NFS (Network File System).

CDR (Common Data Representation — CORBA/OMG): Usado por CORBA. Define una representación binaria compacta. Soporta tipos primitivos y estructurados. El receptor realiza la conversión si es necesario (el emisor indica su *byte order* en el encabezado del mensaje). CDR es un formato eficiente y ampliamente implementado.

Serialización Java: El entorno Java provee un mecanismo integrado de serialización de objetos. Los objetos que implementan la interfaz `Serializable` pueden convertirse en streams de bytes para su transmisión. La JVM gestiona automáticamente la conversión. Desventaja: solo interopera con otras JVMs (problema de portabilidad entre lenguajes).

Marshalling con XML y JSON: Los formatos de texto como XML y JSON son legibles por humanos e independientes del lenguaje. Son el estándar en web services (REST, SOAP). Sus ventajas son la interoperabilidad y la autodescripción; su desventaja es la verbosidad (mayor tamaño del mensaje) y el costo de parseo respecto a formatos binarios.

Protocol Buffers (protobuf) — Google: Formato binario eficiente definido mediante un IDL (Interface Definition Language). El compilador de protobuf genera código de serialización/deserialización en múltiples lenguajes. Muy usado en sistemas modernos por su eficiencia y tipado fuerte.

Proceso completo de marshalling/unmarshalling

[Objeto/estructura en proceso emisor]

↓ marshalling (serialización al formato externo)

[Stream de bytes en formato de red]

↓ transmisión por la red

[Stream de bytes recibido]

↓ unmarshalling (deserialización al formato local)

[Objeto/estructura en proceso receptor]

Compatibilidad de datos entre arquitecturas

Problema	Descripción	Solución
Byte order	Big-endian vs. little-endian	Acordar endianness de red (o indicarlo en header)
Tamaño de tipos	`int` puede ser 16, 32 o 64 bits	Especificar tamaño explícito en el formato
Punto flotante	Distintas representaciones	Usar IEEE 754 como estándar
Strings	ASCII vs. UTF-8 vs. UTF-16	Acordar encoding (generalmente UTF-8)
Alineamiento	Padding entre campos de struct	Empaquetar explícitamente sin padding

7. Virtualización de Red y Overlay Networks (Coulouris, Cap. 4.5)

Concepto de Overlay Network

Una **overlay network** (red superpuesta) es una red lógica construida sobre una red física subyacente. Los nodos de la overlay se comunican entre sí como si fueran vecinos directos, aunque físicamente puedan estar separados por múltiples saltos de red.

La overlay abstrae la topología física y permite implementar servicios de comunicación personalizados (enrutamiento, tolerancia a fallos, búsqueda de recursos) sobre infraestructuras existentes (como Internet).

Tipos de Overlay Networks

Overlays no estructuradas (Unstructured)

Los nodos se conectan de forma arbitraria, sin una topología definida. La búsqueda de recursos se hace por flooding (inundar la red con la consulta) o por random walks. Ejemplo: Gnutella.

- **Ventaja**: fácil de construir y mantener ante fallos.
- **Desventaja**: búsqueda costosa en términos de mensajes; no garantiza encontrar el recurso.

Overlays estructuradas (Structured) — DHT

Las redes overlay estructuradas asignan a cada nodo y a cada recurso un identificador en un espacio de claves uniforme. La ubicación de un recurso está determinada por su clave. La topología de la overlay se mantiene para garantizar que cualquier recurso sea encontrable en O(log N) saltos.

La tecnología central es la **DHT (Distributed Hash Table)**. Implementaciones populares:

- **Chord**: topología de anillo de identificadores. Búsqueda en O(log N) saltos.
- **Pastry / Tapestry**: enrutamiento basado en prefijos de identificadores.
- **Kademlia**: usado en BitTorrent y Ethereum.

Overlays de Content Delivery (CDN)

Las **CDN (Content Delivery Networks)** son overlays especializadas en distribuir contenido estático (imágenes, video, archivos) a usuarios finales con baja latencia. Funcionamiento:

1. El contenido se origina en un **origin server**.
2. Los **edge servers** (servidores en el borde, distribuidos geográficamente) almacenan copias cacheadas.
3. El cliente es redirigido al edge server más cercano (via DNS o anycast).
4. Si el edge no tiene el contenido, lo obtiene del origen y lo cachea.

Ejemplos: Akamai, Cloudflare, Amazon CloudFront.

Virtualización de Red

La **virtualización de red** permite crear múltiples redes lógicas independientes sobre la misma infraestructura física. Tecnologías:

- **VPN (Virtual Private Network)**: túnel cifrado sobre Internet público que emula una LAN privada.
- **VLAN (Virtual LAN)**: segmentación lógica dentro de una red física, implementada en switches.
- **SDN (Software-Defined Networking)**: separación del plano de control y el plano de datos; la topología se programa mediante software.
- **NFV (Network Function Virtualization)**: funciones de red (firewall, balanceador) implementadas como software sobre hardware genérico.

8. Comunicación en Grupo

Concepto de Indirección

"All problems in computer science can be solved by another level of indirection." — Roger Needham, Maurice Wilkes y David Wheeler

En los sistemas distribuidos, el concepto de **indirección** se aplica cada vez más a los paradigmas de comunicación. La **comunicación indirecta** se define como la comunicación entre entidades a través de un intermediario, sin que el emisor y el receptor necesiten conocerse directamente.

Sus propiedades principales son:

- **Desacoplamiento espacial**: el remitente no conoce ni necesita conocer la identidad del receptor, y viceversa.
- **Desacoplamiento temporal**: el emisor y el receptor no necesitan existir al mismo tiempo.

Qué es la Comunicación en Grupo

Ofrece un servicio donde **un mensaje enviado a un grupo es entregado a todos sus miembros**. Un **GRUPO** es un conjunto de procesos con una semántica común; la propiedad fundamental es que cuando se envía un mensaje al grupo, todos los miembros deben recibirlo.

Áreas de aplicación:

- Difusión fiable de información a un gran número de clientes.
- Soporte para aplicaciones colaborativas.
- Soporte para sistemas de monitoreo y administración.

Utilidades concretas:

- **Tolerancia a fallos**: servicios replicados.
- **Aumento de prestaciones** mediante réplicas (objetivo: disponibilidad).
- **Ubicación de objetos** en servicios distribuidos.
- **Actualización múltiple** simultánea.

La comunicación a grupos es una abstracción del multicast y puede implementarse sobre **IP multicast** o una **overlay network**.

9. Tipos de Grupos

Por cantidad de participantes en el envío

Uno a Muchos (Multicast): Un único proceso envía mensajes a todos los miembros del grupo. Los grupos se clasifican en:

Tipo	Descripción
Grupo Cerrado	Solo los miembros del grupo pueden enviar mensajes al grupo. Un proceso externo solo puede enviar mensajes a miembros individuales.
Grupo Abierto	Cualquier proceso del sistema puede enviar mensajes al grupo.

A su vez, los grupos cerrados pueden ser:

- **Compañeros (peer)**: todos los procesos son iguales y las decisiones se toman en forma colectiva.
- **Jerárquicos**: existe un proceso coordinador que toma las decisiones.

Muchos a Uno: Múltiples procesos envían mensajes a un único receptor.

Muchos a Muchos: Múltiples procesos envían mensajes a múltiples receptores.

10. Gestión de Membresías

Un sistema de pasaje de mensajes con soporte de comunicación en grupo debe proveer flexibilidad para:

- Crear y borrar grupos dinámicamente.
- Permitir a un proceso agregarse o abandonar un grupo.

Un mecanismo para administrar esto es un **servidor de grupos**, que:

- Administra las membresías.
- Mantiene una base de datos con los grupos existentes y sus miembros.

Ventaja: fácil de implementar. **Desventaja**: punto único de falla.

Solución: manejo distribuido de la membresía, eliminando la dependencia de un servidor centralizado.

11. Mecanismos de Entrega (Delivery)

Cuando se envía un mensaje a un grupo, existen distintas formas de implementar la entrega:

- **Multicast MAC**: entrega a nivel de capa de enlace usando la dirección MAC multicast.
- **Broadcast MAC**: difusión a todos los dispositivos en el segmento de red.
- **Sender conoce a los miembros**: el emisor mantiene la lista de miembros y les envía el mensaje directamente.
- **Múltiples Unicast**: el emisor envía una copia individual del mensaje a cada miembro del grupo.

Una vez recibido por el SO del receptor, un algoritmo decide cuándo entregarlo al proceso. Un mensaje puede: entregarse inmediatamente, colocarse en espera (para respetar el orden), o rechazarse/descartarse (si es un duplicado).

12. Problemas de Implementación: Confiabilidad

Tipos de Fallos

Fallos por Omisión: Un proceso o canal no realiza las acciones que debería:

- *Crash*: el proceso deja de funcionar.
- *Dropping messages*: el medio descarta mensajes.

Fallos Arbitrarios (Bizantinos): Un proceso o canal falla de forma arbitraria e impredecible.

Fallos de Tiempo: Ocurren en sistemas sincrónicos cuando un proceso no cumple sus restricciones temporales.

Propiedades de la Confiabilidad

- **Integridad**: el mensaje recibido es idéntico al enviado.
- **Validez**: todo mensaje enviado eventualmente será entregado.
- **Acuerdo**: si un miembro del grupo recibe el mensaje, todos lo reciben.
- **Atomicidad**: el mensaje se entrega a todos los miembros o a ninguno.

13. Multicast Atómico

El **multicast atómico** garantiza que todos los miembros del grupo reciban los mensajes de forma consistente.

Propiedad fundamental — Todo o Nada: si un mensaje es enviado al grupo y un miembro lo recibe, ese miembro puede estar seguro de que **todos los demás miembros también lo recibirán**.

Implementación: Persistent Log

En el Emisor:

- Escribe el mensaje en su Log persistente.
- Envía el mensaje al grupo.
- Espera el ACK de confirmación.
- Escribe el ACK recibido en el Log.
- Si hay timeout, retransmite el mensaje.

En el Receptor:

- Registra el mensaje recibido en su Log persistente.
- Envía el ACK al emisor.
- Si el receptor cae y se recupera, puede reconstruir el estado desde el Log.

Cada mensaje tiene un **ID único** para identificar duplicados y retransmisiones.

14. Problemas de Implementación: Ordenamiento

El **ordenamiento de mensajes** es crucial porque múltiples nodos trabajan de manera independiente pero colaborativa.

Ordenamiento basado en Tiempos Globales

En un SSDD esto es inviable porque no existe un tiempo global compartido, varios eventos pueden tener la misma marca de tiempo, y los relojes no están perfectamente sincronizados.

Ordenamiento Consistente (Total Order)

Todos los mensajes se entregan a todos los miembros del grupo **en el mismo orden**.

Reglas formales:

- Si un proceso entrega m antes de m', entonces cualquier otro proceso que entregue m' habrá entregado m primero.

Mecanismo de implementación:

- Se asigna un **ID único y totalmente secuenciado** a cada mensaje.
- El receptor entrega un mensaje solo si ha recibido todos los mensajes con un ID menor.

Ordenamiento Causal

Solo se garantiza el orden entre mensajes **relacionados causalmente** (usando marcas de tiempo de Lamport o vectoriales). Mensajes independientes entre sí pueden entregarse en cualquier orden.

Formalmente: si multicast(G, m) → multicast(G, m'), entonces todos los procesos que entreguen m' habrán entregado m primero.

Resumen comparativo de tipos de ordenamiento

Tipo	Garantía	Complejidad
Global (por tiempo)	Orden exacto de envío	Inviable en SSDD
Consistente (Total Order)	Mismo orden en todos los nodos	Alta; requiere coordinación global
Causal	Orden sólo entre mensajes dependientes	Media; usa vectores de relojes lógicos
Sin orden	No garantiza nada	Baja; máximo rendimiento

15. Colas de Mensajes (Message Queues) — Comunicación Indirecta

Las **colas de mensajes** son un paradigma de comunicación indirecta donde los procesos se comunican a través de un intermediario persistente. El emisor coloca mensajes en una cola; el receptor los extrae cuando está listo. El emisor y el receptor no necesitan estar activos simultáneamente.

Propiedades fundamentales

- **Desacoplamiento temporal**: el emisor puede enviar aunque el receptor no esté disponible en ese momento.
- **Persistencia**: los mensajes son almacenados de forma durable en la cola hasta que son consumidos.
- **Entrega garantizada**: el middleware de colas garantiza que los mensajes eventualmente lleguen.
- **Procesamiento asincrónico**: el emisor no bloquea esperando respuesta.

Modelos de colas

Point-to-Point (P2P): Un mensaje enviado a la cola es consumido por exactamente un receptor. Si hay múltiples consumidores en la misma cola, el sistema de colas distribuye los mensajes entre ellos (load balancing natural).

Productor → [Cola] → Consumidor único

Publish/Subscribe sobre colas: Los mensajes se publican en un *topic*. Todos los suscriptores activos reciben una copia. Cada suscriptor tiene su propia cola lógica de mensajes pendientes.

Publicador → [Topic/Cola] → Consumidor A

→ Consumidor B

→ Consumidor C

Operaciones básicas

Operación	Descripción
`send(queue, message)`	Inserta un mensaje en la cola. Retorna inmediatamente (no bloquea).
`receive(queue)`	Extrae el primer mensaje disponible. Bloquea si la cola está vacía.
`poll(queue)`	Extrae el primer mensaje si hay uno; retorna vacío si no. No bloquea.
`notify(queue, callback)`	Registra un callback que se invoca cuando llega un mensaje.

Garantías de entrega

- **At-most-once**: el mensaje se entrega 0 o 1 veces. Nunca se duplica, pero puede perderse.
- **At-least-once**: el mensaje se entrega 1 o más veces. Nunca se pierde, pero puede duplicarse. El receptor debe ser idempotente.
- **Exactly-once**: el mensaje se entrega exactamente una vez. Requiere coordinación más compleja (2PC o mecanismos de deduplicación).

Tecnologías de colas de mensajes

Sistema	Modelo	Características destacadas
RabbitMQ	AMQP (P2P y Pub/Sub)	Routing flexible, acknowledgments, dead letter queues
Apache Kafka	Pub/Sub con log distribuido	Alta throughput, retención de mensajes, replay

Amazon SQS	P2P	Managed service, at-least-once, FIFO opcional
IBM MQ	Enterprise messaging	Transaccional, alta confiabilidad, JMS compatible
ActiveMQ	AMQP, STOMP, OpenWire	Open source, JMS, múltiples protocolos

Comparación: Colas vs. RPC

Aspecto	RPC	Cola de Mensajes
Acoplamiento temporal	Fuerte (ambos deben estar activos)	Débil (asincrónico)
Respuesta	Síncrona	Asincrónica (callback o polling)
Tolerancia a fallos	Baja (falla si el servidor no responde)	Alta (mensajes persisten)
Escalabilidad	Limitada por el servidor	Alta (múltiples consumidores)
Complejidad	Baja	Media
Caso de uso	Operaciones transaccionales inmediatas	Procesamiento diferido, workflows

16. Enfoques de Memoria Compartida Distribuida

Los **enfoques de memoria compartida** en sistemas distribuidos intentan proveer la abstracción de memoria compartida sobre una red, donde físicamente no existe tal cosa.

Memoria Compartida Distribuida (DSM)

La **DSM (Distributed Shared Memory)** crea la ilusión de un único espacio de memoria compartida sobre nodos que físicamente solo tienen memoria local. El sistema de DSM gestiona transparentemente la coherencia: cuando un proceso lee una dirección de memoria, el sistema recupera el valor del nodo que lo tiene (si no es local).

Ventaja: los programadores pueden usar primitivas de memoria compartida (como en un multiprocesador) sin preocuparse por la comunicación de red.

Desventaja: la coherencia de caché distribuida es costosa. El rendimiento degrada significativamente cuando los patrones de acceso son intensivos en escritura o no tienen localidad.

Modelos de consistencia en DSM:

- **Consistencia estricta**: cualquier lectura devuelve el valor más reciente escrito. Imposible de implementar eficientemente en la práctica.
- **Consistencia secuencial** (Lamport): todos los procesos ven las operaciones en el mismo orden. Es el modelo más fuerte práctico.
- **Consistencia causal**: operaciones relacionadas causalmente son vistas en el mismo orden; operaciones concurrentes pueden verse en diferente orden.
- **Consistencia eventual**: garantiza que si no hay escrituras, eventualmente todos los nodos convergen al mismo valor. La más débil y eficiente; base de muchos sistemas NoSQL.

Espacios de Tuplas (Tuple Spaces)

Un **espacio de tuplas** es un repositorio compartido de tuplas (registros estructurados con campos tipados). Los procesos pueden insertar, leer y extraer tuplas del espacio de forma asociativa (sin direcciones, sino por patrones de contenido).

El modelo fue propuesto por David Gelernter (1985) como el lenguaje **Linda**. Sus operaciones básicas son:

Operación	Descripción
`out(tuple)`	Inserta una tupla en el espacio compartido. No bloquea.
`in(pattern)`	Extrae una tupla que coincida con el patrón. Bloquea hasta que exista.
`rd(pattern)`	Lee (sin extraer) una tupla que coincida. Bloquea hasta que exista.
`inp(pattern)`	Como `in`, pero retorna inmediatamente (null si no hay coincidencia).
`rdp(pattern)`	Como `rd`, pero no bloquea.

Ejemplo: si el espacio contiene `("sensor", 42, "temperatura")`, un proceso puede extraerla con `in("sensor", ?, "temperatura")` donde `?` es un comodín.

Características del modelo:

- **Desacoplamiento total**: emisor y receptor no se conocen; solo comparten el espacio.
- **Comunicación asociativa**: no hay direcciones, sino patrones de datos.
- **Persistencia implícita**: las tuplas permanecen en el espacio hasta que son consumidas.

Implementaciones modernas:

- **JavaSpaces** (Jini): implementación Java del modelo de tuplas para sistemas distribuidos.
- **GigaSpaces**: plataforma comercial de in-memory data grid basada en espacios de tuplas.
- **Redis**: aunque no es puro tuplas, sus estructuras de datos y pub/sub proveen funcionalidad similar.

Comparación de enfoques de memoria compartida

Enfoque	Abstracción	Acoplamiento	Consistencia	Caso de uso
DSM	Espacio de direcciones compartido	Medio	Configurable	Migración de apps paralelas
Tuple Space	Repositorio asociativo	Mínimo	Eventual / por operación	Coordinación asincrónica
Shared-nothing + mensajes	Sin memoria compartida	Explícito	Según protocolo	La mayoría de SSDD modernos

17. Consideraciones Finales

La **comunicación indirecta** es el paradigma adecuado para la difusión de eventos en sistemas donde los receptores pueden ser desconocidos o susceptibles de cambiar dinámicamente. Su capacidad de desacoplar espacialmente y temporalmente a emisores y receptores la hace especialmente útil en arquitecturas de microservicios, sistemas de monitoreo y aplicaciones colaborativas.

La **representación externa de datos** y el **marshalling** son la infraestructura invisible que hace posible que sistemas en diferentes lenguajes, arquitecturas y plataformas se comuniquen correctamente.

Las **overlay networks** proveen la flexibilidad de construir topologías lógicas adaptadas a los requisitos del sistema sobre infraestructuras existentes.

La **confiabilidad** y el **ordenamiento** son propiedades imprescindibles para lograr **consistencia** en el estado global del sistema, y su implementación correcta es uno de los desafíos centrales del diseño de sistemas distribuidos.